



東北大學

机器学习与 Python 实践

第 1 次作业

使用线性回归构建房价预测模型

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 05 月 08 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	算法实现	3
3	实验结果与可扩展内容	4
3.1	实验结果	4
3.2	参数调优及可扩展内容	7
4	讨论与结论	8
4.1	结果讨论	8
4.2	结论总结	8
5	心得体会与思考	9
5.1	个人心得	9
5.2	启发与思考	9
6	附录	9
	算法代码及其注释	9
	前三次参数更新过程	13

1 前言及问题概述

线性回归是最简单、最经典的回归模型之一。假设输入样本 x 为 d 维特征向量，预测目标 y 为连续型取值，线性回归模型可写为

$$\hat{y} = w_0 + w_1x_1 + \cdots + w_dx_d. \quad (1)$$

其中 w_0 为截距项， $w_1, \dots, w_d$ 为各特征对应的回归系数。模型训练的核心任务是根据训练数据估计参数，使预测值与真实值之间的误差尽可能小。

本次实验围绕波士顿房价数据集构建房价预测模型。数据集中每个样本对应一个地区的住房及环境指标，目标变量为房屋价格中位数。实验使用 Python 及 `sklearn.linear_model` 模块中的 `LinearRegression` 函数建立最小二乘线性回归模型，同时结合自定义批量梯度下降模型、随机梯度下降模型、岭回归、Lasso 回归和二次多项式回归进行对比。通过均方误差（MSE）和决定系数 R^2 评价模型效果，并分析归一化、测试集比例和学习率等因素对模型性能的影响。

2 相关基础知识及方法

本章从线性回归原理、数据集字段含义和算法实现流程三个方面说明实验方法。报告正文遵循模板文件中的章节目录，具体实验内容依据原“报告 1”中的代码、运行结果和分析文字整理。

2.1 相关原理

线性回归假设目标变量与输入特征之间存在近似线性关系。设训练集为 $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ ，其中 $x^{(i)} \in \mathbb{R}^d$ ，加入截距项后可记为矩阵形式

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}. \quad (2)$$

最小二乘法通过最小化残差平方和估计模型参数：

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2. \quad (3)$$

当 $\mathbf{X}^T\mathbf{X}$ 可逆时，正规方程可直接求得闭式解：

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}. \quad (4)$$

在 `sklearn.linear_model.LinearRegression` 中，回归系数即由最小二乘思想进行估计。与之相比，梯度下降方法不直接求闭式解，而是沿损失函数负梯度方向迭代更新参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w}), \quad (5)$$

其中 η 为学习率。批量梯度下降每轮使用全部训练样本计算梯度，随机梯度下降每次或每小批次使用部分样本更新参数，适合样本量较大或在线学习场景。

本实验采用两个常用评价指标。均方误差 MSE 越小，表示预测值与真实值越接近；决定系数 R^2 越接近 1，表示模型对目标变量波动的解释能力越强。

2.2 数据描述

波士顿房价数据集共包含 506 条样本，特征数为 13，目标变量为 MEDV，表示自住房屋价格中位数。实验代码从本地 `boston_house_prices.csv` 文件读取数据，并要求目标列名为 MEDV。主要特征含义见表 1。

表 1: 波士顿房价数据集主要字段说明

字段	类型	含义
CRIM	连续特征	城镇人均犯罪率
ZN	连续特征	大面积住宅用地比例
INDUS	连续特征	非零售商业用地比例
CHAS	二值特征	是否邻近查尔斯河
NOX	连续特征	一氧化氮浓度
RM	连续特征	每户平均房间数
AGE	连续特征	1940 年以前建成住房比例
DIS	连续特征	到就业中心的加权距离
RAD	连续特征	到放射状公路的可达性指数
TAX	连续特征	房产税率
PTRATIO	连续特征	城镇师生比例
B	连续特征	人口结构相关指标
LSTAT	连续特征	低收入人口比例
MEDV	目标变量	自住房屋价格中位数

由于各特征量纲差异较大，例如犯罪率、税率、房间数和距离的数值范围并不一致，因此实验在训练集上计算均值和标准差，对训练集及测试集进行 Z-score 标准化。该处理对正规方程求解影响较小，但对梯度下降类方法的收敛速度和稳定性影响明显。

2.3 算法实现

实验流程如下：

- 1. 使用 `pandas` 读取 `boston_house_prices.csv`，将 MEDV 作为预测目标，其余列作为输入特征。

2. 使用 `train_test_split` 按不同测试集比例划分训练集和测试集，主要比较 `test_rate=0.45` 与 `test_rate=0.2`。
3. 对训练集计算均值和标准差，并用同一组统计量标准化训练集和测试集。
4. 分别训练自定义批量梯度下降模型、`LinearRegression` 正规方程模型和 `SGDRegressor` 随机梯度下降模型。
5. 输出训练时长、训练集 MSE、测试集 MSE、训练集 R^2 与测试集 R^2 。
6. 绘制损失曲线、真实值与预测值散点图，并进一步测试学习率、正则化模型和多项式特征对预测效果的影响。

核心实现中，自定义模型的前向传播为

$$\hat{y} = Xw + b, \quad (6)$$

梯度为

$$\frac{\partial J}{\partial w} = \frac{1}{n} X^T (\hat{y} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}). \quad (7)$$

随后按学习率进行参数更新。完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 实验结果

在 `test_rate=0.45` 且进行标准化的情况下，三种线性模型均能较好拟合房价数据，测试集 R^2 约为 0.73–0.74。具体结果见表 2。

表 2: 标准化后 `test_rate=0.45` 的模型结果

模型	训练时长/s	训练 MSE	训练 R^2	测试 MSE	测试 R^2
自定义批量梯度下降	0.0128	22.93	0.73	22.38	0.73
<code>LinearRegression</code>	0.0101	22.60	0.74	21.87	0.74
<code>SGDRegressor</code>	0.0011	22.67	0.73	22.13	0.73

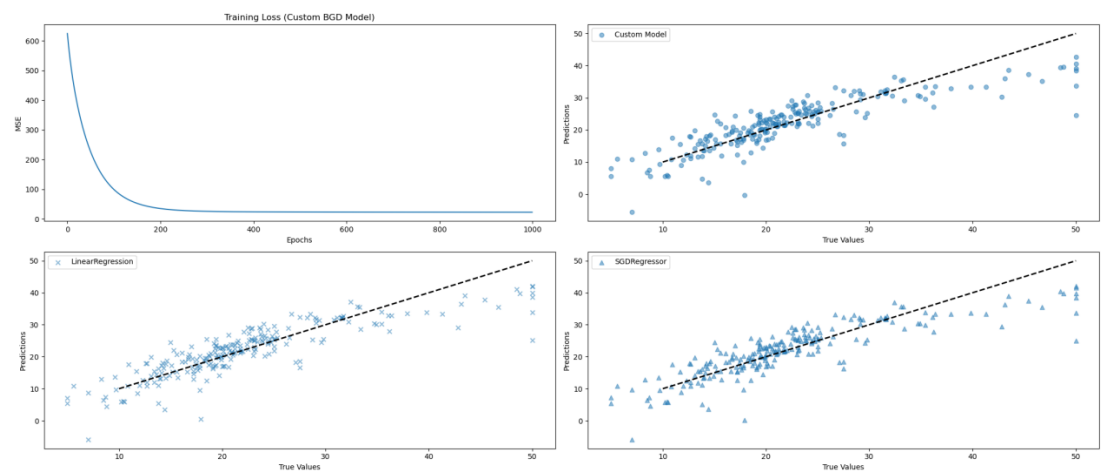


图 1: 标准化后损失曲线及三种模型预测效果

从图 1 可以看到，自定义批量梯度下降模型的损失值在训练前期快速下降，随后逐渐趋于稳定；三个模型的预测散点大体围绕理想预测线分布，说明线性模型能够捕捉房价与主要特征之间的基本关系。其中 `LinearRegression` 的测试集 MSE 最低， R^2 最高，作为最小二乘闭式求解模型具有稳定优势。

将标准化处理去除后，正规方程模型的结果几乎不变，而随机梯度下降模型训练时间明显增加，测试效果略有下降，见表 3。

表 3: 未标准化时的模型结果

模型	训练时长/s	测试 MSE	测试 R^2
<code>LinearRegression</code>	0.0008	21.87	0.74
<code>SGDRegressor</code>	1.7360	23.24	0.72

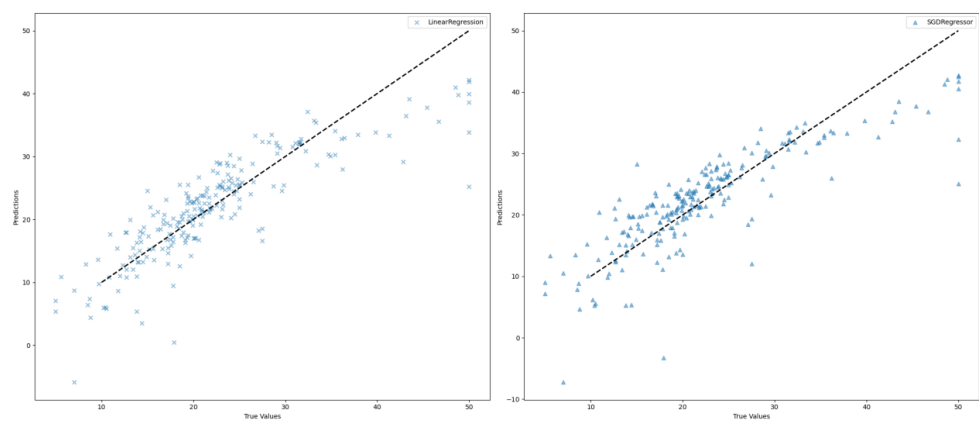


图 2: 未标准化时正规方程与随机梯度下降预测效果

进一步改变学习率为 0.1 时，随机梯度下降出现明显不收敛现象，测试集 MSE 变

得极大， R^2 为大幅负值。这说明梯度下降对学习率敏感，过大的学习率会导致参数更新越过最优区域，造成震荡甚至发散。

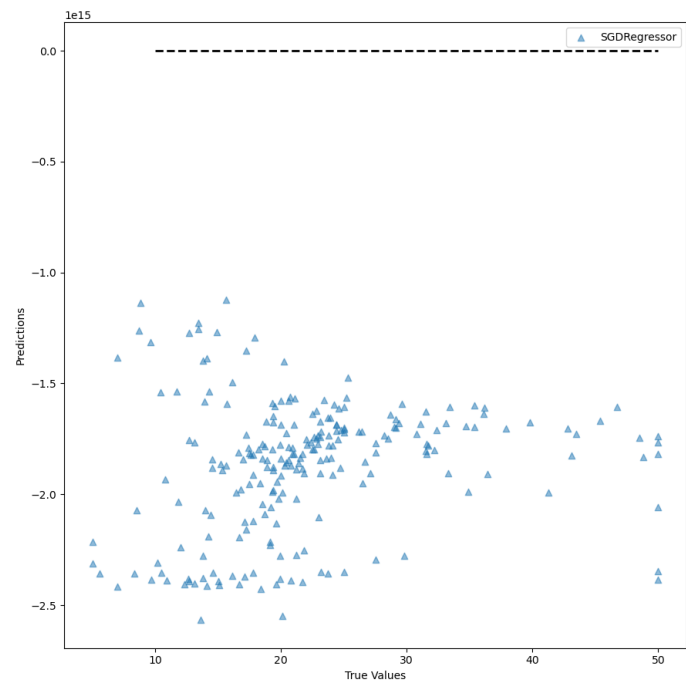


图 3: 学习率过大时随机梯度下降预测发散

当测试集比例设为 0.2 时，训练集 MSE 略低，但测试集 MSE 增大，测试集 R^2 降至约 0.65–0.67，见表 4。本次划分下，`test_rate=0.45` 的测试表现更好。

表 4: 标准化后 `test_rate=0.2` 的模型结果

模型	训练时长/s	训练 MSE	训练 R^2	测试 MSE	测试 R^2
自定义批量梯度下降	0.0138	21.95	0.75	25.64	0.65
<code>LinearRegression</code>	0.0005	21.64	0.75	24.29	0.67
<code>SGDRegressor</code>	0.0010	21.72	0.75	24.88	0.66

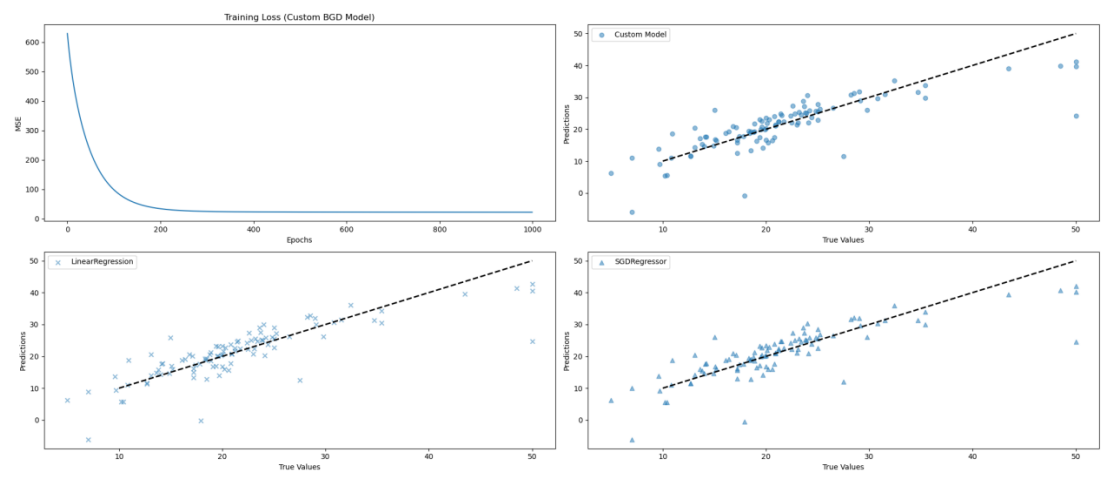


图 4: test_rate=0.2 时损失曲线及预测效果

3.2 参数调优及可扩展内容

在基础线性回归之外，报告 1 中还加入了岭回归、Lasso 回归和二次多项式回归进行扩展实验，结果见表 5。

表 5: 扩展模型测试结果

模型	训练时长/s	测试 MSE	测试 R^2
岭回归	0.0014	24.31	0.67
Lasso 回归	0.0006	25.66	0.65
二次多项式回归	0.0030	14.29	0.81

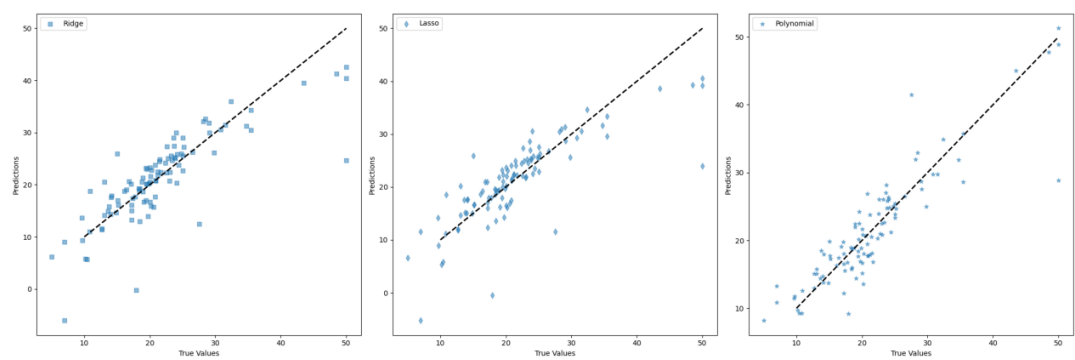


图 5: 岭回归、Lasso 回归与二次多项式回归预测效果

结果显示，在本次实验设定下，二次多项式回归通过引入特征之间的非线性组合，测试集 MSE 降至 14.29， R^2 提升到 0.81，优于基础线性模型。但多项式特征会增加模型复杂度，若阶数继续升高，可能出现过拟合。因此模型扩展时应结合交叉验证、正则化和测试集表现综合判断。

此外，报告 1 对 BGD、SGD 和 MBGD 三种梯度下降策略进行了比较。测试集结果为：BGD 的 MSE 为 16.8523， R^2 为 0.7687；SGD 的 MSE 为 16.0348， R^2 为 0.7799；MBGD 的 MSE 为 16.7107， R^2 为 0.7707。该结果表明，小批量或随机更新在合适学习率下也能取得较好效果。

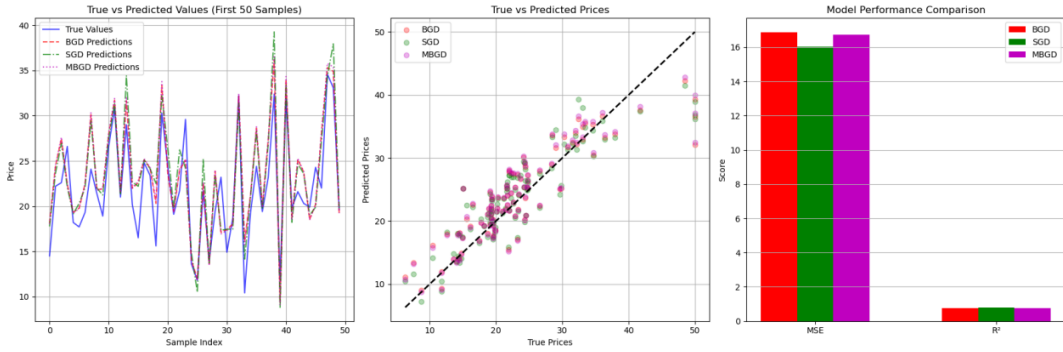


图 6: BGD、SGD 与 MBGD 的预测及指标对比

4 讨论与结论

4.1 结果讨论

通过实验可以得到以下结论。第一，最小二乘线性回归在波士顿房价预测任务中可以作为有效基线模型。在 `test_rate=0.45` 且标准化的条件下，`LinearRegression` 测试集 MSE 为 21.87，测试集 R^2 为 0.74，说明模型能解释约 74% 的房价波动。

第二，正规方程和梯度下降在理论上都可以求解线性回归参数，但计算方式不同。正规方程直接求闭式解，适合特征维度不高、样本规模适中的问题；梯度下降通过迭代逼近最优解，更适合大规模数据或需要在线更新的任务。本实验数据规模较小，因此 `LinearRegression` 的速度和稳定性表现较好。

第三，标准化对梯度下降影响明显。未标准化时，不同特征尺度差异会使损失函数等高线变得狭长，导致梯度下降更新方向不稳定，必须使用更小学习率才能收敛；而标准化使各特征处于相近尺度，优化路径更加平滑。

第四，学习率是梯度下降中的关键超参数。学习率过小会导致收敛缓慢，学习率过大则会导致震荡或发散。本实验中将随机梯度下降学习率调到 0.1 后，模型预测结果明显异常，验证了学习率调节的重要性。

4.2 结论总结

本次实验完成了基于波士顿房价数据集的线性回归建模与预测。实验先使用最小二乘线性回归建立房价预测模型，再与自定义批量梯度下降、随机梯度下降及若干扩展模型进行对比。结果表明，基础线性回归具有实现简单、速度快、解释性强等优点，

适合作为回归任务的入门模型和基线模型；在存在非线性关系时，多项式特征可以进一步提升预测效果，但也需要注意控制模型复杂度。

综合来看，线性回归虽然形式简单，但完整实验过程包含数据读取、训练测试划分、特征标准化、模型训练、指标评价和可视化分析等机器学习基本环节。通过该案例可以较系统地理解监督学习回归任务的基本流程。

5 心得体会与思考

5.1 个人心得

通过本次实验，我对线性回归模型的理解从公式推导进一步落实到代码实现。最小二乘法可以直接给出回归系数，而梯度下降则通过不断迭代更新参数逐步减小误差。两种方法得到的模型形式相同，但适用场景和对数据预处理的敏感程度不同。

在编写程序和整理结果时，我也意识到评价模型不能只看训练集表现。训练集误差低并不代表模型泛化能力强，测试集 MSE 和 R^2 更能反映模型对未知样本的预测效果。可视化散点图也很有帮助，它能直观展示预测点是否围绕理想直线分布。

5.2 启发与思考

线性回归模型的优势在于结构清晰、解释性强，但它对特征与目标之间的关系形式有较强假设。当真实关系存在明显非线性时，仅靠线性模型可能难以充分拟合。后续可以从三个方向继续改进：一是进行特征工程，构造更有意义的组合特征；二是使用交叉验证选择更稳定的测试结果；三是尝试岭回归、Lasso、多项式回归或树模型等方法，在模型复杂度和泛化能力之间取得平衡。

6 附录

算法代码及其注释

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression, SGDRegressor, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from timeit import default_timer
```

```
def load_data(test_rate):
    """Load Boston housing data from a local CSV file."""
    df = pd.read_csv("boston_house_prices.csv")
    if "MEDV" not in df.columns:
        raise ValueError("CSV file must contain a MEDV column as target.")

    X = df.drop("MEDV", axis=1).values.astype(np.float64)
    y = df["MEDV"].values.astype(np.float64)
    return train_test_split(X, y, test_size=test_rate, random_state=42)

def standardize(X_train, X_test):
    mean = X_train.mean(axis=0)
    std = X_train.std(axis=0)
    std[std == 0] = 1
    return (X_train - mean) / std, (X_test - mean) / std

class LinearRegressionNumpy:
    def __init__(self, num_features):
        np.random.seed(42)
        self.w = np.random.randn(num_features)
        self.b = 0.0

    def forward(self, X):
        return X.dot(self.w) + self.b

    @staticmethod
    def loss(y_pred, y_true):
        return np.mean((y_pred - y_true) ** 2)

    @staticmethod
    def gradient(X, y_true, y_pred):
        n = X.shape[0]
        dw = X.T @ (y_pred - y_true) / n
        db = np.mean(y_pred - y_true)
        return dw, db

    def update(self, dw, db, eta):
```

```

        self.w -= eta * dw
        self.b -= eta * db

def train_bgd(self, X_train, y_train, epochs=1000, eta=0.01):
    losses = []
    for _ in range(epochs):
        y_pred = self.forward(X_train)
        losses.append(self.loss(y_pred, y_train))
        dw, db = self.gradient(X_train, y_train, y_pred)
        self.update(dw, db, eta)
    return losses

def evaluate(name, model, X_train, y_train, X_test, y_test):
    start = default_timer()
    model.fit(X_train, y_train)
    elapsed = default_timer() - start
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    print(f"{name}:")
    print(f"  time: {elapsed:.4f}s")
    print(
        f"  train MSE={mean_squared_error(y_train, train_pred):.2f}, "
        f"R2={r2_score(y_train, train_pred):.2f}"
    )
    print(
        f"  test  MSE={mean_squared_error(y_test, test_pred):.2f}, "
        f"R2={r2_score(y_test, test_pred):.2f}"
    )
    return test_pred

def run_experiment(test_rate=0.45):
    X_train, X_test, y_train, y_test = load_data(test_rate)
    X_train_scaled, X_test_scaled = standardize(X_train, X_test)

    custom_model = LinearRegressionNumpy(X_train_scaled.shape[1])
    start = default_timer()
    losses = custom_model.train_bgd(X_train_scaled, y_train)
    custom_time = default_timer() - start
    custom_train_pred = custom_model.forward(X_train_scaled)

```

```

custom_test_pred = custom_model.forward(X_test_scaled)

print(f"test rate = {test_rate}")
print("custom BGD:")
print(f"  time: {custom_time:.4f}s")
print(
    f"  train MSE={mean_squared_error(y_train, custom_train_pred):.2f}, "
    f"R2={r2_score(y_train, custom_train_pred):.2f}"
)
print(
    f"  test  MSE={mean_squared_error(y_test, custom_test_pred):.2f}, "
    f"R2={r2_score(y_test, custom_test_pred):.2f}"
)

lr_pred = evaluate(
    "sklearn LinearRegression",
    LinearRegression(),
    X_train_scaled,
    y_train,
    X_test_scaled,
    y_test,
)

sgd_pred = evaluate(
    "sklearn SGDRegressor",
    SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.01,
        ↪ random_state=42),
    X_train_scaled,
    y_train,
    X_test_scaled,
    y_test,
)

plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.plot(losses)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE")

for idx, (pred, label, marker) in enumerate(

```

```

        [(custom_test_pred, "Custom BGD", "o"), (lr_pred,
        ↪ "LinearRegression", "x"), (sgd_pred, "SGDRegressor", "^)],
        start=2,
    ):
        plt.subplot(2, 2, idx)
        plt.scatter(y_test, pred, marker=marker, alpha=0.5, label=label)
        plt.plot([10, 50], [10, 50], "k--", lw=2)
        plt.xlabel("True Values")
        plt.ylabel("Predictions")
        plt.legend()

    plt.tight_layout()
    plt.savefig("RESULT.png", dpi=180)
    return X_train_scaled, X_test_scaled, y_train, y_test

def run_extensions(X_train, X_test, y_train, y_test):
    ridge = Ridge(alpha=1.0)
    lasso = Lasso(alpha=0.1, max_iter=10000)
    poly = PolynomialFeatures(degree=2, include_bias=False)

    evaluate("Ridge", ridge, X_train, y_train, X_test, y_test)
    evaluate("Lasso", lasso, X_train, y_train, X_test, y_test)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)
    evaluate("Polynomial degree=2", LinearRegression(), X_train_poly,
    ↪ y_train, X_test_poly, y_test)

if __name__ == "__main__":
    data = run_experiment(test_rate=0.45)
    run_extensions(*data)

```

前三次参数更新过程

报告 1 中记录的前三次批量梯度下降参数更新如下：

迭代 1：
梯度向量：

```
[ 3.5548 -3.2920 4.4314 -1.6078 3.9072 -6.3888 3.4429 -2.2753 3.4938  
↪ 4.2933 4.6557 -3.0425 6.7698 -22.5340]
```

权重变化向量:

```
[ -0.0355 0.0329 -0.0443 0.0161 -0.0391 0.0639 -0.0344 0.0228 -0.0349  
↪ -0.0429 -0.0466 0.0304 -0.0677 0.2253]
```

当前权重向量:

```
[ -0.0409 0.0416 -0.0346 0.0195 -0.0490 0.0587 -0.0474 0.0319 -0.0332  
↪ -0.0354 -0.0525 0.0489 -0.0495 0.2241]
```

当前损失值: 592.1959

迭代 2:

梯度向量:

```
[ 3.3399 -3.0786 4.1422 -1.5844 3.6319 -6.1818 3.1906 -2.0254 3.2288  
↪ 4.0095 4.4563 -2.8697 6.4819 -22.3087]
```

权重变化向量:

```
[ -0.0334 0.0308 -0.0414 0.0158 -0.0363 0.0618 -0.0319 0.0203 -0.0323  
↪ -0.0401 -0.0446 0.0287 -0.0648 0.2231]
```

当前权重向量:

```
[ -0.0743 0.0724 -0.0760 0.0353 -0.0854 0.1205 -0.0793 0.0522 -0.0655  
↪ -0.0755 -0.0970 0.0776 -0.1143 0.4472]
```

当前损失值: 577.6884

迭代 3:

梯度向量:

```
[ 3.1382 -2.8785 3.8709 -1.5611 3.3742 -5.9852 2.9544 -1.7919 2.9804  
↪ 3.7432 4.2678 -2.7075 6.2104 -22.0856]
```

权重变化向量:

```
[ -0.0314 0.0288 -0.0387 0.0156 -0.0337 0.0599 -0.0295 0.0179 -0.0298  
↪ -0.0374 -0.0427 0.0271 -0.0621 0.2209]
```

当前权重向量:

```
[ -0.1057 0.1012 -0.1147 0.0509 -0.1191 0.1803 -0.1089 0.0701 -0.0953  
↪ -0.1129 -0.1397 0.1046 -0.1765 0.6680]
```

当前损失值: 563.8403